



**UNIVERSIDAD NACIONAL
MAYOR DE SAN MARCOS**

**Facultad de Ingeniería de Sistemas e Informática
Escuela Profesional de Ingeniería de Sistemas**

**PROGRAMACIÓN DE
COMPUTADORAS II**
**Sistema de Riego
Automatizado**

Docente:

❖ Guerra Grados, Luis Angel

Integrantes:

- Hurtado Flores, Italo Josué
- Salazar Ruiz, Angel Eduardo Daniel
- Sachahuaman Jaramillo, Alejandro
- Villegas Torres, Miguel Angel

Lima, Perú

2025

Índice

1. INTRODUCCIÓN.....	1
2. PROBLEMA.....	1
2.1. Descripción de la realidad.....	1
2.2. Problema general.....	2
3. OBJETIVOS.....	3
3.1. Objetivo general.....	3
3.2. Objetivos específicos.....	3
4. ANÁLISIS DE CLASES SOLUCIÓN.....	3
4.1. Paquete: controlRiegoAutomatizado (Lógica de Negocio).....	4
4.2. Paquete: controlSensores (Simulación de Hardware).....	5
4.3. Paquete: baseDeDatos (Persistencia).....	6
4.4. Paquete: interfacesGraficas (Vista).....	6
5. DIAGRAMA DE CLASES.....	8
6. IMPLEMENTACIÓN DE CLASES.....	8
6.1. Paquete controlRiegoAutomatizado.....	8
6.2. Paquete controlSensores.....	16
6.3. Paquete baseDeDatos.....	17
6.4. Paquete interfacesGraficas.....	21
7. PROGRAMA USANDO POO.....	36
8. INTERFACES GRÁFICAS Y RESULTADOS.....	36
8.1. Ventana de Acceso y Registro (registro.java).....	36
8.2. Gestión y Selección de Cultivos (GestionCultivos.java).....	37
8.3. Reporte de Resultados (VentanaResultados.java).....	38
9. CONECTIVIDAD A BASE DE DATOS.....	38
10. REFERENCIAS.....	41

1. INTRODUCCIÓN

En la actualidad, la gestión eficiente del agua en la agricultura se ha convertido en una prioridad mundial debido al aumento de la demanda alimentaria y a la escasez de recursos hídricos. Tradicionalmente, los sistemas de riego se han basado en métodos manuales simples que no consideran las condiciones ambientales ni las necesidades específicas de cada cultivo, lo que genera desperdicio de agua y baja productividad. En los últimos años, la tendencia global se ha orientado hacia la implementación de sistemas de riego automatizados e inteligentes, capaces de ajustar la cantidad y frecuencia del riego mediante sensores y algoritmos

El presente proyecto propone una solución tecnológica basada en un sistema de riego automatizado que utiliza principios de la programación orientada a objetos para modelar los diferentes componentes del sistema: cultivos, sensores y estrategias de riego. Este enfoque permite aplicar distintos métodos de riego según el tipo de cultivo, región y estación del año, integrando sensores de humedad y temperatura que determinan cuándo y cómo activar el riego.

Con este desarrollo se busca optimizar el uso del agua, aumentar la eficiencia agrícola y reducir la intervención humana, promoviendo una gestión más sostenible y controlada del recurso hídrico. Además, el sistema está diseñado para ser escalable, lo que facilita su integración futura con tecnologías de inteligencia artificial o plataformas de monitoreo remoto. En conjunto, esta propuesta se alinea con las soluciones modernas que se aplican actualmente en la agricultura inteligente a nivel mundial.

2. PROBLEMA

2.1. Descripción de la realidad

En muchas zonas agrícolas de Perú, especialmente en regiones áridas y semiáridas como la costa y algunas partes de la sierra, la gestión eficiente del agua para el riego es un desafío crítico. Esta situación se debe a la escasez del recurso hídrico y a la variabilidad climática.

Actualmente, los agricultores suelen depender de métodos tradicionales de riego que no optimizan el uso del agua. Esta dependencia de métodos obsoletos resulta en un desperdicio del recurso y, consecuentemente, en una menor productividad agrícola.

Además, la falta de sistemas automatizados y de monitoreo en tiempo real impide a los agricultores ajustar los tiempos y volúmenes de riego de manera precisa. Esto afecta negativamente el crecimiento de los cultivos y la sostenibilidad a largo plazo del recurso hídrico.

2.2. Problema general

¿Cómo podría mejorarse la eficiencia del uso del agua en las zonas agrícolas del Perú mediante un sistema automatizado de riego que utiliza sensores de humedad y datos climáticos en tiempo real?

La gestión ineficiente del agua en la agricultura, especialmente en regiones áridas y semiáridas del país, genera desperdicio del recurso hídrico y reduce la productividad de los cultivos. Los métodos tradicionales de riego no permiten un control preciso del volumen y tiempo de aplicación del agua, lo que impacta negativamente en la sostenibilidad ambiental y económica de las actividades agrícolas.

3. OBJETIVOS

3.1. Objetivo general

Desarrollar un programa en Java para el control automatizado y eficiente del riego agrícola, que integre sensores de humedad, datos climáticos y algoritmos de optimización, con el fin de mejorar el uso del agua y asegurar un riego adecuado según el tipo de cultivo y las condiciones ambientales.

3.2. Objetivos específicos

- 3.2.1. Diseñar un sistema que elija la mejor estrategia de riego para los distintos tipos de cultivos que podrían haber en una zona agrícola de cualquier parte de nuestro país.
- 3.2.2. Realizar una correcta relación entre clases para que el sistema funcione correctamente.
- 3.2.3. Emplear conceptos como herencia múltiple e interfaces para hacer el código más limpio.
- 3.2.4. Desarrollar una interfaz gráfica que permita a los agricultores monitorear y controlar el sistema de riego en tiempo real.
- 3.2.5. Evaluar la eficiencia del sistema automatizado en comparación con métodos tradicionales de riego.

4. ANÁLISIS DE CLASES SOLUCIÓN

El software **Sistema de Riego Automatizado** ha sido diseñado bajo una arquitectura modular, organizando el código en cuatro paquetes fundamentales para garantizar el desacoplamiento y la mantenibilidad:

- **controlRiegoAutomatizado**: Núcleo de la lógica de negocio, patrones de diseño y reglas agronómicas.
- **controlSensores**: Simulación de hardware y lectura de variables ambientales.
- **baseDeDatos**: Capa de persistencia y comunicación con MySQL (DAO).
- **interfacesGraficas**: Capa de presentación e interacción con el usuario (GUI).

A continuación, se detalla la función y estructura de cada clase por paquete:

4.1. Paquete: controlRiegoAutomatizado (Lógica de Negocio)

Este paquete contiene el "cerebro" del sistema, implementando el Patrón de Diseño *Strategy* y las entidades principales.

Clase / Interfaz	Descripción	Atributos / Componentes Principales
Cultivo	Entidad principal que representa la planta a ser gestionada. Coordina sus propios sensores y ejecuta la estrategia asignada.	tipo, region, estacion, misSensores (Lista), estrategia
GeneradorDeEstrategias	Clase encargada de la toma de decisiones (Árbol de decisión). Evalúa la demanda hídrica y el estrés ambiental para asignar una estrategia.	Constantes de evaluación (DEMANDA_ALTA, ESTRES_ALTO, etc.)
Region (Enum)	Enumeración que define las 8 regiones naturales del Perú con sus respectivos rangos climáticos predefinidos.	nombre, tempMin, tempMax
EstrategiaRiego	Interfaz que define el contrato común para todos los algoritmos de riego (Patrón <i>Strategy</i>).	Método abstracto aplicar()

EstrategiaRiegoBasico	Implementación concreta para cultivos de baja demanda o condiciones estables.	N/A (Solo implementación lógica)
EstrategiaRiegoEficiente	Implementación concreta optimizada para situaciones de estrés medio o ahorro de agua.	N/A (Solo implementación lógica)
EstrategiaRiegoEspecializado	Implementación concreta para cultivos sensibles o condiciones climáticas extremas.	N/A (Solo implementación lógica)
SistemaDeRiego	Clase administradora que gestiona la colección de cultivos añadidos antes del procesamiento.	listaCultivos, generador

4.2. Paquete: controlSensores (Simulación de Hardware)

Encargado de la abstracción de los dispositivos de medición, utilizando herencia para especializar los tipos de sensores.

Clase	Descripción	Atributos Principales
Sensor (Abstracta)	Clase padre que define el comportamiento base de cualquier sensor y su vinculación a una región específica.	id, lecturaActual, region, random

SensorTemperatura	Especialización que simula la lectura térmica respetando los límites climáticos de la región (Enum).	Heredados de Sensor
SensorHumedad	Especialización que simula la lectura higrométrica del suelo (0-100%).	Heredados de Sensor

4.3. Paquete: baseDeDatos (Persistencia)

Maneja todas las transacciones SQL, separando la conexión de la lógica de acceso a datos (Patrón DAO).

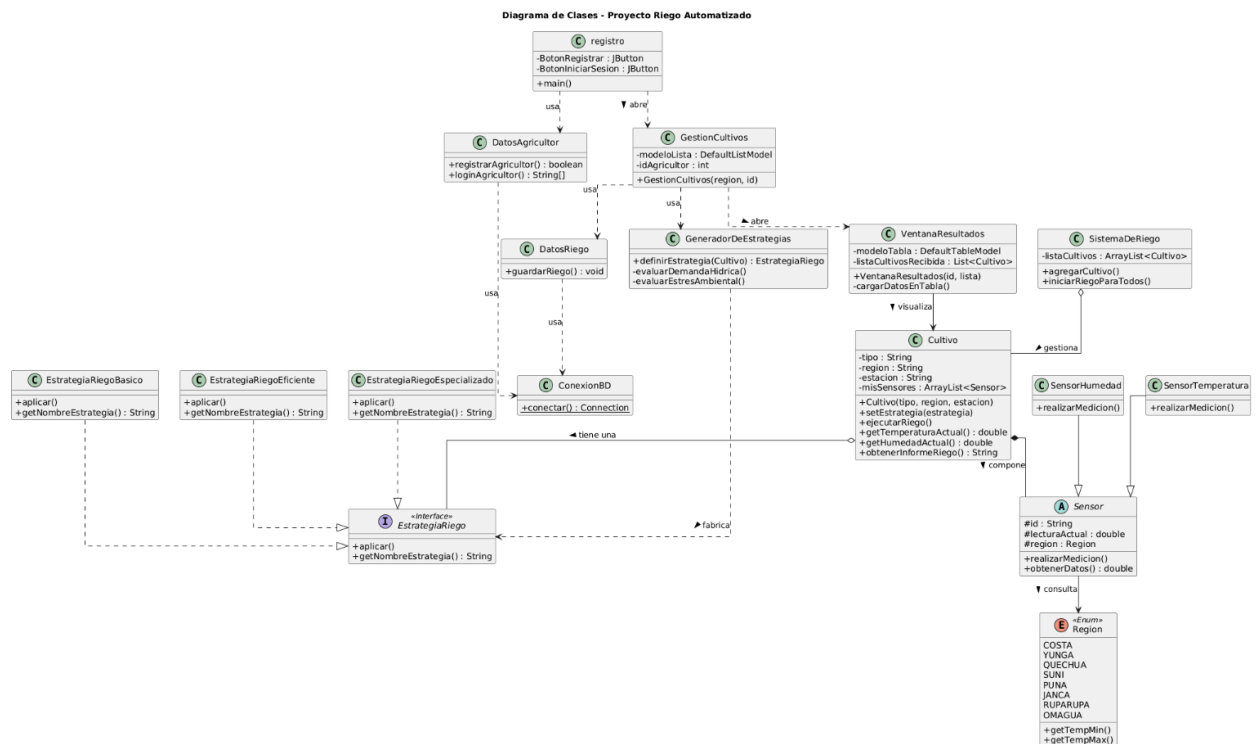
Clase	Descripción	Atributos Principales
ConexionBD	Gestiona la conexión JDBC con el servidor MySQL (Driver y Credenciales).	url, user, password, driver
DatosAgricultor	DAO (Data Access Object) encargado del registro de usuarios, validación de DNI único e inicio de sesión.	N/A (Métodos de conexión)
DatosRiego	DAO encargado de almacenar el historial de riegos procesados, vinculándolos al ID del agricultor.	N/A (Métodos de conexión)

4.4. Paquete: interfacesGraficas (Vista)

Implementación visual utilizando Java Swing, diseñada para guiar al usuario a través del flujo del sistema.

Clase	Descripción	Atributos Principales
registro	Ventana inicial. Permite el registro de nuevos agricultores o el inicio de sesión mediante DNI.	TextoDNI, ComboRegion, BotonRegistrar, BotonIniciarSesion
GestionCultivos	Panel principal de gestión. Permite añadir múltiples cultivos a una lista temporal antes del análisis.	modeloLista, idAgricultor, regionAgricultor, TextoCultivo
VentanaResultados	Pantalla final de reporte. Muestra una tabla detallada con los datos sensoriales y la estrategia aplicada tras el procesamiento.	modeloTabla (JTable), listaCultivosRecibida, BotonRegarSeleccionado

5. DIAGRAMA DE CLASES



6. IMPLEMENTACIÓN DE CLASES

6.1. Paquete controlRiegoAutomatizado

6.1.1. Clase Cultivo

Java

package controlRiegoAutomatizado;

import java.util.ArrayList;

import controlSensores.*;

```

public class Cultivo {
    private String tipo;
    private String region;
    private String estacion;
    private EstrategiaRiego estrategia;
    private ArrayList<Sensor> misSensores;
  
```

```

    public Cultivo(String tipo, String region, String estacion) {
        this.tipo = tipo;
        this.region = region;
        this.estacion = estacion;
  
```

```

        this.misSensores = new ArrayList<>();
        instalarSensores();
    }

    private void instalarSensores() {
        Region regionEnumSeleccionada = null;
        for (Region r : Region.values()) {
            if (r.getNombre().equalsIgnoreCase(this.region)) {
                regionEnumSeleccionada = r;
                break;
            }
        }
        if (regionEnumSeleccionada == null) {
            regionEnumSeleccionada = Region.COSTA;
        }

        misSensores.add(new SensorTemperatura("TEMP-001", regionEnumSeleccionada));
        misSensores.add(new SensorHumedad("HUM-001", regionEnumSeleccionada));
    }

    public void setEstrategia(EstrategiaRiego estrategia) {
        this.estrategia = estrategia;
    }

    public void ejecutarRiego() {
        System.out.println("\n--- Analizando cultivo: " + tipo + " en " + region + " ---");
        System.out.println(" (Sensores analizando el entorno...)");
        for (Sensor s : misSensores) {
            s.realizarMedicion();
        }

        if (this.estrategia != null) {
            estrategia.aplicar();
        }
    }

    public String getTipo() {
        return tipo;
    }

    public String getRegion() {
        return region;
    }

    public String getEstacion() {
        return estacion;
    }

```

```

public double getTemperaturaActual() {
    for (Sensor s : misSensores) {
        if (s instanceof SensorTemperatura) {
            return s.obtenerDatos();
        }
    }
    return 0.0;
}

public double getHumedadActual() {
    for (Sensor s : misSensores) {
        if (s instanceof SensorHumedad) {
            return s.obtenerDatos();
        }
    }
    return 0.0;
}

public String obtenerInformeRiego() {
    double temp = getTemperaturaActual();
    double humed = getHumedadActual();
    String estrat = (estrategia != null) ? estrategia.getNombreEstrategia() : "Sin estrategia";

    return "=== REPORTE DE RIEGO ===\n\n" +
        "Cultivo: " + tipo + "\n" +
        "Región: " + region + "\n" +
        "Estación: " + estacion + "\n" +
        "-----\n" +
        "CONDICIONES DETECTADAS:\n" +
        "-> Temperatura: " + String.format("%.2f", temp) + " °C\n" +
        "-> Humedad Suelo: " + String.format("%.2f", humed) + " %\n" +
        "-----\n" +
        "ACCIÓN TOMADA:\n" +
        "-> " + estrat + "\n\n" +
        "Estado: Riego Ejecutado Exitosamente.";
}

public String getNombreEstrategia() {
    if (this.estrategia != null) {
        return this.estrategia.getNombreEstrategia();
    }
    return "Sin Asignar";
}
}

```

6.1.2. Interfaz EstrategiaRiego

```
Java
package controlRiegoAutomatizado;

public interface EstrategiaRiego {
    public void aplicar();
    public String getNombreEstrategia();
}
```

6.1.3. Clase EstrategiaRiegoBásico

```
Java
package controlRiegoAutomatizado;

public class EstrategiaRiegoBásico implements EstrategiaRiego {

    @Override
    public void aplicar() {
        System.out.println("...Cultivo analizado! Iniciando riego básico...");
        System.out.println("--- RIEGO BÁSICO COMPLETADO ---");
    }

    @Override
    public String getNombreEstrategia() {
        return "Riego Básico";
    }
}
```

6.1.4. Clase EstrategiaRiegoEficiente

```
Java
package controlRiegoAutomatizado;

public class EstrategiaRiegoEficiente implements EstrategiaRiego {

    @Override
    public void aplicar() {
        System.out.println("...Cultivo analizado!\n --> Aplicando algoritmos de optimización...");
        System.out.println("Iniciando riego eficiente...");
        System.out.println("--- RIEGO EFICIENTE COMPLETADO ---");
    }
}
```

```

@Override
public String getNombreEstrategia() {
    return "Riego Eficiente";
}
}

```

6.1.5. Clase EstrategiaRiegoEspecializado

Java

```
package controlRiegoAutomatizado;
```

```
public class EstrategiaRiegoEspecializado implements EstrategiaRiego {
```

```

@Override
public void aplicar() {
    System.out.println("...Cultivo analizado!");
    System.out.println("--> Utilizando sensores de humedad...\n--> Aplicando algoritmos de
optimización...");
    System.out.println("Iniciando riego especializado...");
    System.out.println("--- RIEGO ESPECIALIZADO COMPLETADO ---");
}

```

```

@Override
public String getNombreEstrategia() {
    return "Riego Especializado";
}
}

```

6.1.6. Clase GeneradorDeEstrategias

Java

```
package controlRiegoAutomatizado;
```

```

public class GeneradorDeEstrategias {

    private static final String DEMANDA_ALTA = "ALTA";
    private static final String DEMANDA_MEDIA = "MEDIA";
    private static final String DEMANDA_BAJA = "BAJA";

    private static final String ESTRES_ALTO = "ALTO";
    private static final String ESTRES_MEDIO = "MEDIO";
    private static final String ESTRES_BAJO = "BAJO";
}

```

```

public EstrategiaRiego definirEstrategia(Cultivo cult) {
    String demandaHidrica = evaluarDemandaHidrica(cult.getTipo());
    double temperatura = cult.getTemperaturaActual();
    double humedad = cult.getHumedadActual();
    String nivelEstres = evaluarEstresAmbiental(temperatura, humedad);

    System.out.println(" [Análisis] Planta: " + demandaHidrica + " | Clima: " + nivelEstres);

    if (demandaHidrica.equals(DEMANDA_ALTA)) {
        if (nivelEstres.equals(ESTRES_ALTO) || nivelEstres.equals(ESTRES_MEDIO)) {
            return new EstrategiaRiegoEspecializado();
        } else {
            return new EstrategiaRiegoEficiente();
        }
    }
    else if (demandaHidrica.equals(DEMANDA_MEDIA)) {
        if (nivelEstres.equals(ESTRES_ALTO)) {
            return new EstrategiaRiegoEspecializado();
        } else if (nivelEstres.equals(ESTRES_MEDIO)) {
            return new EstrategiaRiegoEficiente();
        } else {
            return new EstrategiaRiegoBásico();
        }
    }
    else {
        if (nivelEstres.equals(ESTRES_ALTO)) {
            return new EstrategiaRiegoEficiente();
        } else {
            return new EstrategiaRiegoBásico();
        }
    }
}

private String evaluarDemandaHidrica(String nombreCultivo) {
    String nombre = nombreCultivo.toLowerCase();
    if (nombre.contains("arroz") || nombre.contains("maiz") || nombre.contains("maíz") ||
nombre.contains("caña")) {
        return DEMANDA_ALTA;
    }
    else if (nombre.contains("cactus") || nombre.contains("tuna") ||
nombre.contains("suculenta")) {
        return DEMANDA_BAJA;
    }
    else {
        return DEMANDA_MEDIA;
    }
}

```

```

    }
}

private String evaluarEstresAmbiental(double temp, double hum) {
    if (temp > 25 && hum < 40) {
        return ESTRES_ALTO;
    }
    else if (temp < 15 || hum > 80) {
        return ESTRES_BAJO;
    }
    else {
        return ESTRES_MEDIO;
    }
}
}

```

6.1.7. Enum Region

Java

```
package controlRiegoAutomatizado;
```

```

public enum Region {
    COSTA("Costa o chala", 15.0, 28.0),
    YUNGA("Yunga", 20.0, 25.0),
    QUECHUA("Quechua", 11.0, 16.0),
    SUNI("Suni", 7.0, 10.0),
    PUNA("Puna", 0.0, 7.0),
    JANCA("Janca", -20.0, 0.0),
    RUPARUPA("Rupa Rupa", 22.0, 25.0),
    OMAGUA("Omagua", 24.0, 32.0);

    private final String nombre;
    private final double tempMin;
    private final double tempMax;

    private Region(String nombre, double tempMin, double tempMax) {
        this.nombre = nombre;
        this.tempMin = tempMin;
        this.tempMax = tempMax;
    }

    public String getNombre() {
        return nombre;
    }
}

```



```

public double getTempMin() {
    return tempMin;
}
public double getTempMax() {
    return tempMax;
}

public static Region buscarPorNombre(String texto) {
    for (Region r : Region.values()) {
        if (r.getNombre().equalsIgnoreCase(texto)) {
            return r;
        }
    }
    return null;
}
}

```

6.1.8. Clase SistemaDeRiego

Java

```
package controlRiegoAutomatizado;
```

```
import java.util.ArrayList;
```

```

public class SistemaDeRiego {
    private GeneradorDeEstrategias generador;
    private ArrayList<Cultivo> listaCultivos;

    public SistemaDeRiego() {
        generador = new GeneradorDeEstrategias();
        listaCultivos = new ArrayList<>();
    }

    public void agregarCultivo(String tipo, String region, String estacion) {
        Cultivo cultivoNuevo = new Cultivo(tipo, region, estacion);
        EstrategiaRiego estrategiaEspecificas = generador.definirEstrategia(cultivoNuevo);
        cultivoNuevo.setEstrategia(estrategiaEspecificas);
        listaCultivos.add(cultivoNuevo);
    }

    public void iniciarRiegoParaTodos() {
        for (Cultivo c : listaCultivos) {
            c.ejecutarRiego();
        }
    }
}

```

```
}  
}
```

6.2. Paquete controlSensores

6.2.1. Clase Abstracta Sensor

```
Java  
package controlSensores;  
  
import controlRiegoAutomatizado.Region;  
import java.util.Random;  
  
public abstract class Sensor {  
    protected String id;  
    protected Random random;  
    protected double lecturaActual;  
    protected Region region;  
  
    public Sensor(String id, Region region) {  
        this.id = id;  
        this.region = region;  
        this.random = new Random();  
    }  
  
    public abstract void realizarMedicion();  
  
    public double obtenerDatos() {  
        return lecturaActual;  
    }  
}
```

6.2.2. Clase SensorHumedad

```
Java  
package controlSensores;  
  
import controlRiegoAutomatizado.Region;  
  
public class SensorHumedad extends Sensor {  
  
    public SensorHumedad(String id, Region region) {  
        super(id, region);  
    }  
}
```

```

    }

    @Override
    public void realizarMedicion() {
        lecturaActual = random.nextDouble() * 100;
    }
}

```

6.2.3. Clase SensorTemperatura

```

Java
package controlSensores;

import controlRiegoAutomatizado.Region;

public class SensorTemperatura extends Sensor {

    public SensorTemperatura(String id, Region region) {
        super(id, region);
    }

    @Override
    public void realizarMedicion() {
        double min = region.getTempMin();
        double max = region.getTempMax();
        double diferencia = max - min;
        double temperaturaDetectada = (random.nextDouble() * diferencia) + min;
        this.lecturaActual = temperaturaDetectada;
    }
}

```

6.3. Paquete baseDeDatos

6.3.1. Clase ConexionBD

```

Java
package baseDeDatos;

import java.sql.*;

public class ConexionBD {
    private static final String url = "jdbc:mysql://localhost/sistema_riego_bd";
}

```

```

private static final String user = "root";
private static final String password = "";
private static final String driver = "com.mysql.cj.jdbc.Driver";

public static Connection conectar() {
    Connection conexion = null;
    try {
        Class.forName(driver);
        conexion = DriverManager.getConnection(url, user, password);
        System.out.println("<< ¡Conexión exitosa a la base de datos! >>");
    } catch(ClassNotFoundException e) {
        System.out.println("¡Error! Falta el Driver JDBC de MySQL en el proyecto.");
    } catch(SQLException e) {
        System.out.println("¡Error SQL! No se pudo conectar. Verifica usuario/clave.");
    }
    return conexion;
}
}

```

6.3.2. Clase DatosAgricultor

```

Java
package baseDeDatos;

import java.sql.*;
import javax.swing.JOptionPane;

public class DatosAgricultor {
    public boolean registrarAgricultor(String dni, String nombre, String apellido, String region) {
        Connection conexion = null;
        PreparedStatement psVerificar = null;
        PreparedStatement psInsertar = null;
        ResultSet rs = null;

        try {
            conexion = ConexionBD.conectar();

            String sqlVerificar = "SELECT id_agricultor FROM agricultor WHERE dni = ?";
            psVerificar = conexion.prepareStatement(sqlVerificar);
            psVerificar.setString(1, dni);
            rs = psVerificar.executeQuery();

            if (rs.next()) {

```

```

        JOptionPane.showMessageDialog(null, "El DNI " + dni + " ya está registrado.\nUse el
botón 'Iniciar Sesión'.");
        return false;
    }

```

```

    String sqlInsertar = "INSERT INTO agricultor (dni, nombre, apellido, region) VALUES
(?, ?, ?, ?)";

```

```

    psInsertar = conexion.prepareStatement(sqlInsertar);
    psInsertar.setString(1, dni);
    psInsertar.setString(2, nombre);
    psInsertar.setString(3, apellido);
    psInsertar.setString(4, region);

```

```

    psInsertar.executeUpdate();
    return true;

```

```

    } catch (SQLException e) {
        System.err.println("Error al registrar: " + e.getMessage());
        return false;
    } finally {

```

```

        try {
            if (rs != null) rs.close();
            if (psVerificar != null) psVerificar.close();
            if (psInsertar != null) psInsertar.close();
            if (conexion != null) conexion.close();
        } catch (SQLException e) {}
    }
}

```

```

public String[] loginAgricultor(String dni) {

```

```

    Connection conexion = null;
    PreparedStatement ps = null;
    ResultSet rs = null;

```

```

    String sql = "SELECT id_agricultor, nombre, region FROM agricultor WHERE dni = ?";
    String[] datosEncontrados = null;

```

```

    try {
        conexion = ConexionBD.conectar();
        ps = conexion.prepareStatement(sql);
        ps.setString(1, dni);
        rs = ps.executeQuery();

```

```

        if (rs.next()) {
            String id = String.valueOf(rs.getInt("id_agricultor"));

```

```

String nombre = rs.getString("nombre");
String region = rs.getString("region");

datosEncontrados = new String[] { id, nombre, region };
OptionPane.showMessageDialog(null, "¡Bienvenido, " + nombre + "!");
}
} catch (SQLException e) {
    System.err.println("Error al loguear: " + e.getMessage());
} finally {
    try {
        if (rs != null) rs.close();
        if (ps != null) ps.close();
        if (conexion != null) conexion.close();
    } catch (SQLException e) {
        System.out.println("Error cerrando conexión");
    }
}
return datosEncontrados;
}
}

```

6.3.3. Clase DatosRiego

```

Java
package baseDeDatos;

import java.sql.*;

public class DatosRiego {
    public void guardarRiego(int idAgricultor, String cultivo, double temp, double hum, String
    estrategia) {
        Connection conexion = null;
        PreparedStatement ps = null;

        String sql = "INSERT INTO historial_riego " +
            "(id_agricultor, nombre_cultivo, temp_detectada, humedad_detectada,
            estrategia_aplicada) " +
            "VALUES (?, ?, ?, ?, ?)";

        try {
            conexion = ConexionBD.conectar();
            ps = conexion.prepareStatement(sql);

            ps.setInt(1, idAgricultor);

```

```

        ps.setString(2, cultivo);
        ps.setDouble(3, temp);
        ps.setDouble(4, hum);
        ps.setString(5, estrategia);

        ps.executeUpdate();
        System.out.println(">> Riego guardado en historial para: " + cultivo);

    } catch (SQLException e) {
        System.err.println("Error al guardar historial: " + e.getMessage());
    } finally {
        try {
            if (ps != null) ps.close();
            if (conexion != null) conexion.close();
        } catch (SQLException e) {}
    }
}
}
}

```

6.4. Paquete interfacesGraficas

6.4.1. Clase registro

Java

```
package interfacesGraficas;
```

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import baseDeDatos.DatosAgricultor;

```

```
public class registro extends JFrame {
```

```

    private JPanel jPanel1;
    private JLabel jLabel1, jLabel2, jLabel3, jLabel5;
    private JButton BotonLimpiar, BotonRegistrar, BotonIniciarSesion;
    private JComboBox<String> ComboRegion;
    private JLabel LabelNombre, LabelApellido, LabelRegion, LabelDNI, LabelRegistro;
    private JTextField TextoDNI, TextoApellido, TextoNombre;

```

```

    public registro() {
        initComponents();
        setLocationRelativeTo(null);
    }

```

```
}
```

```
private void initComponents() {  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setTitle("Registro de Agricultor");  
    setResizable(false);  
  
    jPanel1 = new JPanel();  
    jPanel1.setLayout(null);  
  
    LabelRegistro = new JLabel("REGISTRO DE AGRICULTOR");  
    LabelRegistro.setFont(new Font("Roboto", Font.BOLD, 48));  
    LabelRegistro.setForeground(new Color(99, 58, 39));  
    LabelRegistro.setBounds(290, 40, 700, 60);  
    jPanel1.add(LabelRegistro);  
  
    LabelDNI = new JLabel("DNI");  
    LabelDNI.setFont(new Font("Roboto", Font.BOLD, 36));  
    LabelDNI.setForeground(Color.WHITE);  
    LabelDNI.setBounds(180, 190, 100, 40);  
    jPanel1.add(LabelDNI);  
  
    LabelNombre = new JLabel("NOMBRE");  
    LabelNombre.setFont(new Font("Roboto", Font.BOLD, 36));  
    LabelNombre.setForeground(Color.WHITE);  
    LabelNombre.setBounds(180, 260, 200, 40);  
    jPanel1.add(LabelNombre);  
  
    LabelApellido = new JLabel("APELLIDO");  
    LabelApellido.setFont(new Font("Roboto", Font.BOLD, 36));  
    LabelApellido.setForeground(Color.WHITE);  
    LabelApellido.setBounds(180, 330, 200, 40);  
    jPanel1.add(LabelApellido);  
  
    LabelRegion = new JLabel("REGIÓN");  
    LabelRegion.setFont(new Font("Roboto", Font.BOLD, 36));  
    LabelRegion.setForeground(Color.WHITE);  
    LabelRegion.setBounds(180, 400, 200, 40);  
    jPanel1.add(LabelRegion);  
  
    Font fuenteTexto = new Font("Roboto", Font.PLAIN, 28);  
    javax.swing.border.Border paddingInterno = BorderFactory.createEmptyBorder(0, 15, 0, 0);  
  
    TextoDNI = new JTextField("");  
    TextoDNI.setFont(fuenteTexto);
```



```
TextoDNI.setForeground(Color.GRAY);
TextoDNI.setBorder(paddingInterno);
TextoDNI.setBounds(390, 190, 230, 40);
jPanel1.add(TextoDNI);
```

```
TextoNombre = new JTextField("");
TextoNombre.setFont(fuenteTexto);
TextoNombre.setForeground(Color.GRAY);
TextoNombre.setBorder(paddingInterno);
TextoNombre.setBounds(390, 260, 300, 40);
jPanel1.add(TextoNombre);
```

```
TextoApellido = new JTextField("");
TextoApellido.setFont(fuenteTexto);
TextoApellido.setForeground(Color.GRAY);
TextoApellido.setBorder(paddingInterno);
TextoApellido.setBounds(390, 330, 300, 40);
jPanel1.add(TextoApellido);
```

```
String[] regiones = { "Costa o Chala", "Yunga", "Quechua", "Suní", "Puna", "Janca", "Rupa  
Rupa", "Omagua" };
ComboRegion = new JComboBox<>(regiones);
ComboRegion.setFont(fuenteTexto);
ComboRegion.setForeground(Color.GRAY);
ComboRegion.setBackground(Color.WHITE);
ComboRegion.setBounds(390, 400, 240, 40);
jPanel1.add(ComboRegion);
```

```
Color colorBotones = new Color(85, 45, 45);
```

```
BotonRegistrar = new JButton("Registrar");
BotonRegistrar.setBackground(colorBotones);
BotonRegistrar.setFont(new Font("Segoe UI", Font.PLAIN, 36));
BotonRegistrar.setForeground(Color.WHITE);
BotonRegistrar.setBounds(230, 510, 180, 50);
BotonRegistrar.addActionListener(this::BotonRegistrarActionPerformed);
jPanel1.add(BotonRegistrar);
```

```
BotonLimpiar = new JButton("Limpiar");
BotonLimpiar.setBackground(colorBotones);
BotonLimpiar.setFont(new Font("Segoe UI", Font.PLAIN, 36));
BotonLimpiar.setForeground(Color.WHITE);
BotonLimpiar.setBounds(450, 510, 160, 50);
BotonLimpiar.addActionListener(this::BotonLimpiarActionPerformed);
jPanel1.add(BotonLimpiar);
```

```

    BotonIniciarSesion = new JButton("Iniciar Sesión");
    BotonIniciarSesion.setBackground(colorBotones);
    BotonIniciarSesion.setFont(new Font("Segoe UI", Font.PLAIN, 36));
    BotonIniciarSesion.setForeground(Color.WHITE);
    BotonIniciarSesion.setBounds(650, 510, 250, 50);
    BotonIniciarSesion.addActionListener(this::BotonIniciarSesionActionPerformed);
    jPanel1.add(BotonIniciarSesion);

    jLabel2 = new JLabel();
    try { jLabel2.setIcon(new
ImageIcon(getClass().getResource("/imagenesParaGUI/planta.JPG"))); }
    catch (Exception e) {}
    jLabel2.setBounds(90, 30, 170, 150);
    jPanel1.add(jLabel2);

    jLabel5 = new JLabel();
    try { jLabel5.setIcon(new
ImageIcon(getClass().getResource("/imagenesParaGUI/ecorregioness.JPG"))); }
    catch (Exception e) {}
    jLabel5.setBounds(720, 170, 320, 300);
    jPanel1.add(jLabel5);

    jLabel3 = new JLabel();
    try { jLabel3.setIcon(new
ImageIcon(getClass().getResource("/imagenesParaGUI/fonfomarron.JPG"))); }
    catch (Exception e) {}
    jLabel3.setBounds(160, 130, 730, 370);
    jPanel1.add(jLabel3);

    jLabel1 = new JLabel();
    try { jLabel1.setIcon(new
ImageIcon(getClass().getResource("/imagenesParaGUI/fondo.JPG"))); }
    catch (Exception e) {}
    jLabel1.setBounds(0, 0, 1090, 580);
    jPanel1.add(jLabel1);

    setPreferredSize(new Dimension(1100, 620));
    getContentPane().add(jPanel1);
    pack();
}

private void BotonRegistrarActionPerformed(ActionEvent evt) {
    String dni = TextoDNI.getText();
    String nombre = TextoNombre.getText();

```

```

String apellido = TextoApellido.getText();
String region = (String) ComboRegion.getSelectedItem();

if (dni.isEmpty() || nombre.isEmpty() || apellido.isEmpty() || dni.equals("Ingrese su DNI")) {
    JOptionPane.showMessageDialog(this, "Por favor, complete todos los campos.");
    return;
}

DatosAgricultor dao = new DatosAgricultor();
boolean exito = dao.registrarAgricultor(dni, nombre, apellido, region);

if (exito) {
    JOptionPane.showMessageDialog(this, "¡Registro Exitoso!");
    String[] datosNuevos = dao.loginAgricultor(dni);
    int idNuevo = Integer.parseInt(datosNuevos[0]);
    GestionCultivos ventanaSiguiente = new GestionCultivos(region, idNuevo);
    ventanaSiguiente.setVisible(true);
    this.dispose();
}
}

private void BotonLimpiarActionPerformed(ActionEvent evt) {
    TextoDNI.setText("");
    TextoNombre.setText("");
    TextoApellido.setText("");
    ComboRegion.setSelectedIndex(0);
}

private void BotonIniciarSesionActionPerformed(ActionEvent evt) {
    String dniIngresado = JOptionPane.showInputDialog(this, "Ingrese su DNI para entrar:");

    if (dniIngresado != null && !dniIngresado.trim().isEmpty()) {

        DatosAgricultor dao = new DatosAgricultor();
        String[] datosUsuario = dao.loginAgricultor(dniIngresado);

        if (datosUsuario != null) {
            int idRecuperado = Integer.parseInt(datosUsuario[0]);
            String regionRecuperada = datosUsuario[2];
            GestionCultivos ventanaSiguiente = new GestionCultivos(regionRecuperada,
idRecuperado);
            ventanaSiguiente.setVisible(true);
            this.dispose();
        } else {

```

```

        JOptionPane.showMessageDialog(this, "Error: DNI no encontrado.\nPor favor
regístrese primero.");
    }
}

public static void main(String args[]) {
    try {
        for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (Exception ex) { ex.printStackTrace(); }

    EventQueue.invokeLater(() -> new registro().setVisible(true));
}
}

```

6.4.2. Clase GestionCultivos

Java

```
package interfacesGraficas;
```

```

import javax.swing.*;
import javax.swing.border.LineBorder;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.util.ArrayList;
import controlRiegoAutomatizado.*;
import baseDeDatos.DatosRiego;

```

```
public class GestionCultivos extends JFrame {
```

```

    private static final long serialVersionUID = 1L;
    private DefaultListModel<String> modeloLista;
    private String regionAgricultor;
    private int idAgricultor;

```

```

    private JPanel jPanel1;
    private JButton BotonSiguiente, BotonLimpiar, BotonAñadir, BotonBorrarTodo;
    private JList<String> jList1;
    private JScrollPane scrollLista;

```

```

private JLabel LabelCultivo, LabelRegistro, LabelSubtituloLista, JLabelFondo,
jLabelPanelMarron, JLabelLogo;
private JTextField TextoCultivo;

private final Color COLOR_VERDE = new Color(106, 153, 43);
private final Color COLOR_ROJO = new Color(180, 60, 60);
private final Color COLOR_TEXTO_OSCURO = new Color(60, 60, 60);

public GestionCultivos(String region, int id) {
    this.regionAgricultor = region;
    this.idAgricultor = id;

    initComponents();
    this.setLocationRelativeTo(null);
    this.setTitle("Gestión de Cultivos - Región: " + region + " (ID: " + id + ")");
}

private void initComponents() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setResizable(false);

    modeloLista = new DefaultListModel<>();

    jPanel1 = new JPanel();
    jPanel1.setLayout(null);

    JLabel sombraTitulo = new JLabel("ANÁLISIS DE CULTIVOS",
SwingConstants.CENTER);
    sombraTitulo.setFont(new Font("Segoe UI", Font.BOLD, 42));
    sombraTitulo.setForeground(new Color(99, 58, 39));
    sombraTitulo.setBounds(103, 23, 780, 60);
    jPanel1.add(sombraTitulo);

    LabelRegistro = new JLabel("ANÁLISIS DE CULTIVOS", SwingConstants.CENTER);
    LabelRegistro.setFont(new Font("Segoe UI", Font.BOLD, 42));
    LabelRegistro.setForeground(new Color(225, 205, 170));
    LabelRegistro.setBounds(100, 20, 780, 60);
    jPanel1.add(LabelRegistro);

    LabelCultivo = new JLabel("CULTIVO:", SwingConstants.RIGHT);
    LabelCultivo.setFont(new Font("Segoe UI", Font.BOLD, 28));
    LabelCultivo.setForeground(Color.WHITE);
    LabelCultivo.setBounds(180, 150, 150, 40);
    jPanel1.add(LabelCultivo);

```

```

TextoCultivo = new JTextField();
TextoCultivo.setFont(new Font("Segoe UI", Font.PLAIN, 24));
TextoCultivo.setForeground(COLOR_TEXTO_OSCURO);
TextoCultivo.setBorder(BorderFactory.createCompoundBorder(
    new LineBorder(COLOR_VERDE, 2),
    BorderFactory.createEmptyBorder(5, 10, 5, 10)));
TextoCultivo.setBounds(350, 150, 300, 45);
TextoCultivo.addActionListener(this::TextoCultivoActionPerformed);
jPanel1.add(TextoCultivo);

BotonAñadir = crearBotonModerno("Añadir", COLOR_VERDE);
BotonAñadir.setBounds(350, 220, 140, 45);
BotonAñadir.addActionListener(this::BotonAñadirActionPerformed);
jPanel1.add(BotonAñadir);

BotonLimpiar = crearBotonModerno("Limpiar", COLOR_ROJO);
BotonLimpiar.setBounds(510, 220, 140, 45);
BotonLimpiar.addActionListener(this::BotonLimpiarActionPerformed);
jPanel1.add(BotonLimpiar);

LabelSubtituloLista = new JLabel("Lista de cultivos a analizar (Selecciona para borrar:)");
LabelSubtituloLista.setFont(new Font("Segoe UI", Font.PLAIN, 14));
LabelSubtituloLista.setForeground(Color.WHITE);
LabelSubtituloLista.setBounds(350, 275, 350, 20);
jPanel1.add(LabelSubtituloLista);

jList1 = new JList<>();
jList1.setModel(modeloLista);
jList1.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
jList1.setFont(new Font("Segoe UI", Font.PLAIN, 18));
jList1.setSelectionBackground(new Color(220, 240, 200));
jList1.setSelectionForeground(COLOR_TEXTO_OSCURO);

scrollLista = new JScrollPane(jList1);
scrollLista.setBorder(new LineBorder(COLOR_VERDE, 1));
scrollLista.setBounds(350, 295, 300, 130);
jPanel1.add(scrollLista);

BotonBorrarTodo = crearBotonModerno("Borrar seleccionado", COLOR_ROJO);
BotonBorrarTodo.setFont(new Font("Segoe UI", Font.BOLD, 13));
BotonBorrarTodo.setBounds(700, 370, 160, 45);
BotonBorrarTodo.addActionListener(this::BotonBorrarTodoActionPerformed);
jPanel1.add(BotonBorrarTodo);

BotonSiguiente = crearBotonModerno("Siguiente >", COLOR_VERDE);

```

```

BotonSiguiente.setFont(new Font("Segoe UI", Font.BOLD, 16));
BotonSiguiente.setBounds(700, 430, 160, 50);
BotonSiguiente.addActionListener(this::BotonSiguienteActionPerformed);
jPanel1.add(BotonSiguiente);

jLabelLogo = new JLabel();
cargarImagen(jLabelLogo, "/imagenesParaGUI/planta.JPG");
jLabelLogo.setBounds(40, 30, 170, 140);
jPanel1.add(jLabelLogo);

jLabelPanelMarron = new JLabel();
cargarImagen(jLabelPanelMarron, "/imagenesParaGUI/fonfomarron.JPG");
jLabelPanelMarron.setBounds(100, 30, 780, 470);
jPanel1.add(jLabelPanelMarron);

jLabelFondo = new JLabel();
cargarImagen(jLabelFondo, "/imagenesParaGUI/fondo.JPG");
jLabelFondo.setBounds(0, 0, 1000, 600);
jPanel1.add(jLabelFondo);

setSize(new Dimension(980, 570));
getContentPane().add(jPanel1);
pack();
}

private JButton crearBotonModerno(String texto, Color colorFondo) {
    JButton btn = new JButton(texto);
    btn.setFont(new Font("Segoe UI", Font.BOLD, 18));
    btn.setBackground(colorFondo);
    btn.setForeground(Color.WHITE);
    btn.setFocusPainted(false);
    btn.setBorderPainted(false);
    btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
    return btn;
}

private void cargarImagen(JLabel label, String ruta) {
    try { label.setIcon(new ImageIcon(getClass().getResource(ruta))); }
    catch (Exception e) { label.setText("Error IMG"); }
}

private void TextoCultivoActionPerformed(ActionEvent evt) { BotonAñadir.doClick(); }

private void BotonAñadirActionPerformed(ActionEvent evt) {
    String cultivo = TextoCultivo.getText().trim();

```

```

        if (!cultivo.isEmpty()) {
            modeloLista.addElement(cultivo);
            TextoCultivo.setText("");
            TextoCultivo.requestFocus();
        } else { Toolkit.getDefaultToolkit().beep(); }
    }

    private void BotonLimpiarActionPerformed(ActionEvent evt) {
        TextoCultivo.setText("");
        TextoCultivo.requestFocus();
    }

    private void BotonBorrarTodoActionPerformed(ActionEvent evt) {
        int indiceSeleccionado = jList1.getSelectedIndex();
        if (indiceSeleccionado != -1) {
            modeloLista.remove(indiceSeleccionado);
        } else {
            if (!modeloLista.isEmpty()) JOptionPane.showMessageDialog(this, "Selecciona un cultivo para borrar.");
        }
        TextoCultivo.requestFocus();
    }

    private void BotonSiguienteActionPerformed(ActionEvent evt) {
        if (modeloLista.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Añade al menos un cultivo.", "Aviso",
            JOptionPane.WARNING_MESSAGE);
            return;
        }

        ArrayList<Cultivo> listaProcesada = new ArrayList<>();
        GeneradorDeEstrategias generador = new GeneradorDeEstrategias();
        DatosRiego daoRiego = new DatosRiego();

        for (int i = 0; i < modeloLista.getSize(); i++) {
            String nombreCultivo = modeloLista.getElementAt(i);

            Cultivo miCultivo = new Cultivo(nombreCultivo, regionAgricultor, "Verano");
            EstrategiaRiego estrategia = generador.definirEstrategia(miCultivo);
            miCultivo.setEstrategia(estrategia);
            miCultivo.ejecutarRiego();

            double temp = miCultivo.getTemperaturaActual();
            double hum = miCultivo.getHumedadActual();
            String nomEst = estrategia.getNombreEstrategia();

```



```

        daoRiego.guardarRiego(idAgricultor, nombreCultivo, temp, hum, nomEst);
        listaProcesada.add(miCultivo);
    }

    VentanaResultados ventanaFinal = new VentanaResultados(idAgricultor, listaProcesada);
    ventanaFinal.setVisible(true);
    this.dispose();
}

public static void main(String args[]) {
    try { UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()); }
    catch (Exception ex) {}
    EventQueue.invokeLater(() -> new GestionCultivos("Prueba", 1).setVisible(true));
}
}

```

6.4.3. Clase VentanaResultados

Java

```
package interfacesGraficas;
```

```

import javax.swing.*;
import javax.swing.border.LineBorder;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;

```

```

import controlRiegoAutomatizado.Cultivo;
import controlRiegoAutomatizado.EstrategiaRiego;

```

```
public class VentanaResultados extends JFrame {
```

```
    private static final long serialVersionUID = 1L;
```

```

    private JPanel jPanel1;
    private JLabel LabelTitulo, LabelSombraTitulo, jLabelFondo;
    private JTable jTable1;
    private JScrollPane scrollPaneTabla;
    private JButton BotonVolver, BotonRegarSeleccionado, BotonRegarTodo;

```

```

private DefaultTableModel modeloTabla;

private final Color COLOR_VERDE = new Color(106, 153, 43);
private final Color COLOR_TEXTO_OSCURO = new Color(60, 60, 60);
private final Color COLOR_AZUL_RIEGO = new Color(0, 120, 215);
private final Color COLOR_AZUL_TODO = new Color(0, 80, 180);

private int idAgricultor;
private List<Cultivo> listaCultivosRecibida;

public VentanaResultados(int idAgricultor, List<Cultivo> cultivosProcesados) {
    this.idAgricultor = idAgricultor;
    this.listaCultivosRecibida = cultivosProcesados;

    initComponents();
    this.setLocationRelativeTo(null);
    cargarDatosEnTabla();
}

private void initComponents() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setTitle("Sistema Agrícola - Resultados del Riego");
    setResizable(false);

    jPanel1 = new JPanel();
    jPanel1.setLayout(null);

    LabelSombraTitulo = new JLabel("RESULTADOS DE RIEGO",
SwingConstants.CENTER);
    LabelSombraTitulo.setFont(new Font("Segoe UI", Font.BOLD, 42));
    LabelSombraTitulo.setForeground(new Color(99, 58, 39));
    LabelSombraTitulo.setBounds(52, 52, 880, 60);
    jPanel1.add(LabelSombraTitulo);

    LabelTitulo = new JLabel("RESULTADOS DE RIEGO", SwingConstants.CENTER);
    LabelTitulo.setFont(new Font("Segoe UI", Font.BOLD, 42));
    LabelTitulo.setForeground(new Color(225, 205, 170));
    LabelTitulo.setBounds(50, 50, 880, 60);
    jPanel1.add(LabelTitulo);

    String[] columnas = {
        "ID Agri.",
        "Cultivo",
        "Región",
        "Temp. (°C)",
    }

```

```

        "Humedad (%)",
        "Estrategia Aplicada",
        "Hora Riego"
    };
    modeloTabla = new DefaultTableModel(columnas, 0) {
        @Override public boolean isCellEditable(int row, int column) { return false; }
    };
    jTable1 = new JTable(modeloTabla);
    estilizarTabla(jTable1);

    scrollPaneTabla = new JScrollPane(jTable1);
    scrollPaneTabla.setBorder(new LineBorder(COLOR_VERDE, 2));
    scrollPaneTabla.setBounds(50, 130, 880, 350);
    jPanel1.add(scrollPaneTabla);

    BotonRegarSeleccionado = crearBotonModerno("Ver Informe Detallado",
    COLOR_AZUL_RIEGO);
    BotonRegarSeleccionado.setBounds(90, 500, 220, 45);

    BotonRegarSeleccionado.addActionListener(this::BotonRegarSeleccionadoActionPerformed);
    jPanel1.add(BotonRegarSeleccionado);

    BotonRegarTodo = crearBotonModerno("Finalizar Riego", COLOR_AZUL_TODO);
    BotonRegarTodo.setBounds(330, 500, 180, 45);
    BotonRegarTodo.addActionListener(this::BotonRegarTodoActionPerformed);
    jPanel1.add(BotonRegarTodo);

    BotonVolver = crearBotonModerno("Cerrar Sesión", new Color(180, 60, 60));
    BotonVolver.setBounds(750, 500, 160, 45);
    BotonVolver.addActionListener(this::BotonVolverActionPerformed);
    jPanel1.add(BotonVolver);

    jLabelFondo = new JLabel();
    cargarImagen(jLabelFondo, "/imagenesParaGUI/fondo.JPG");
    jLabelFondo.setBounds(0, 0, 1000, 600);
    jPanel1.add(jLabelFondo);

    setPreferredSize(new Dimension(980, 600));
    getContentPane().add(jPanel1);
    pack();
}

private void cargarDatosEnTabla() {
    modeloTabla.setRowCount(0);
    DateTimeFormatter dtf = DateTimeFormatter.ofPattern("HH:mm:ss");

```

```

String horaActual = LocalDateTime.now().format(dtf);

for (Cultivo c : listaCultivosRecibida) {
    double temp = c.getTemperaturaActual();
    double hum = c.getHumedadActual();
    String estrategiaReal = c.getNombreEstrategia();

    Object[] fila = {
        idAgricultor,
        c.getTipo(),
        c.getRegion(),
        String.format("%.1f", temp),
        String.format("%.1f", hum),
        estrategiaReal,
        horaActual
    };
    modeloTabla.addRow(fila);
}

private void estilizarTabla(JTable table) {
    table.setFont(new Font("Segoe UI", Font.PLAIN, 14));
    table.setRowHeight(35);
    table.setForeground(COLOR_TEXTO_OSCURO);
    table.setSelectionBackground(new Color(180, 220, 255));
    table.setSelectionForeground(Color.BLACK);
    table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

    DefaultTableCellRenderer headerRenderer = new DefaultTableCellRenderer();
    headerRenderer.setBackground(COLOR_VERDE);
    headerRenderer.setForeground(Color.WHITE);
    headerRenderer.setFont(new Font("Segoe UI", Font.BOLD, 14));
    headerRenderer.setHorizontalAlignment(JLabel.CENTER);
    for (int i = 0; i < table.getColumnModel().getColumnCount(); i++)
        table.getColumnModel().getColumn(i).setHeaderRenderer(headerRenderer);

    DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
    centerRenderer.setHorizontalAlignment(JLabel.CENTER);
    table.setDefaultRenderer(Object.class, centerRenderer);
}

private JButton crearBotonModerno(String texto, Color colorFondo) {
    JButton btn = new JButton(texto);
    btn.setFont(new Font("Segoe UI", Font.BOLD, 14));
    btn.setBackground(colorFondo);
}

```

```

        btn.setForeground(Color.WHITE);
        btn.setFocusPainted(false);
        btn.setBorderPainted(false);
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        return btn;
    }

    private void cargarImagen(JLabel label, String ruta) {
        try { label.setIcon(new ImageIcon(getClass().getResource(ruta))); } catch (Exception e) {}
    }

    private void BotonRegarSeleccionadoActionPerformed(ActionEvent evt) {
        int fila = jTable1.getSelectedRow();
        if (fila == -1) {
            JOptionPane.showMessageDialog(this, "Selecciona una fila para ver el informe.");
            return;
        }
        Cultivo cultivo = listaCultivosRecibida.get(fila);
        JOptionPane.showMessageDialog(this, cultivo.obtenerInformeRiego(), "Informe Técnico",
JOptionPane.INFORMATION_MESSAGE);
    }

    private void BotonRegarTodoActionPerformed(ActionEvent evt) {
        JOptionPane.showMessageDialog(this, "Todos los datos han sido guardados en la Base de
Datos Histórica.\nProceso Finalizado.");
        System.exit(0);
    }

    private void BotonVolverActionPerformed(ActionEvent evt) {
        new registro().setVisible(true);
        this.dispose();
    }
}

```

7. PROGRAMA USANDO POO

Departamento	Región Predominante / Capital
Amazonas	Rupa Rupa (Sana Alta)
Ancaash	Quichua (Sana) y Chala (Costa)
Ajuntinas	Quichua
Arequipa	Quichua / Chala
Ayacucho	Quichua
Cajamarca	Quichua
Cusco	Quichua (Capitán) / Selva Alta y Baja
Huancavelica	Sana
Huánuco	Yunga / Puente / Rupa Rupa
Ica	Chala
Jirón	Quichua (Capitán) / Selva
La Libertad	Chala / Capitán / Selva
Lambayeque	Chala
Lima	Chala (Capitán) / Todas las regiones andinas
Loreto	Orongo
Moravia de Dios	Orongo
Moravia	Yunga / Chala
Pasco	Puna (Capitán) / Rupa Rupa
Piura	Chala
Puno	Sana / Puna
San Martín	Rupa Rupa
Tarma	Yunga / Chala
Tumbes	Chala

8. INTERFACES GRÁFICAS Y RESULTADOS

Para la interacción entre el usuario (agricultor) y el sistema, se desarrolló una interfaz gráfica de usuario (GUI) robusta y amigable, utilizando la biblioteca **Java Swing** (javax.swing) y **AWT** (java.awt). El diseño se centró en la usabilidad, empleando una paleta de colores corporativos (tonos tierra y verdes) para mantener una coherencia visual con la temática agrícola.

La capa de "Vista" del patrón MVC se implementó a través del paquete interfacesGraficas, compuesto por tres ventanas secuenciales que guían el flujo de la aplicación:

8.1. Ventana de Acceso y Registro (registro.java)

Es el punto de entrada al sistema. Esta interfaz permite dos funciones críticas: registrar nuevos usuarios en la base de datos y autenticar a los existentes mediante su DNI.

- **Componentes Clave:** Se utilizaron JTextField personalizados con bordes limpios para la entrada de datos y un JComboBox para la selección estricta de la **Región** (dato fundamental para la calibración de sensores).
- **Validación:** El sistema valida que los campos no estén vacíos antes de conectar con la clase DatosAgricultor.
- **Resultado Visual:**

Departamento	Región Predominante / Capital
Amazonas	Rupa Rupa (Selva Alta)
Ancaezh	Quichua (Sierra) y Chala (Costa)
Ayacucho	Quichua
Arequipa	Quichua / Chala
Ayacucho	Quichua
Cajamarca	Quichua
Cusco	Quichua (Capitán) / Selva Alta y Baja
Huanuaycha	Quichua
Huanuco	Yunga (Flore) / Rupa Rupa
Ica	Chala
Jurín	Quichua (Capitán) / Selva
La Libertad	Chala (Capitán) / Sierra
Lambayeque	Chala
Lima	Chala (Capitán) / Todas las regiones andinas
Loreto	Orangut
Madre de Dios	Orangut
Moravia	Yunga / Chala
Pasco	Puna (Capitán) / Rupa Rupa
Piura	Chala
Puno	Quichua / Puna
San Martín	Rupa Rupa
Tarma	Yunga / Chala
Tumbes	Chala

8.2. Gestión y Selección de Cultivos (GestionCultivos.java)

Una vez autenticado, el sistema dirige al usuario a esta ventana, transfiriendo internamente los datos de sesión (ID y Región). Aquí, el agricultor define la lista de cultivos que desea analizar en la campaña actual.

- **Manejo de Listas:** Se implementó un DefaultListModel vinculado a un JList para permitir la adición y eliminación dinámica de cultivos antes del procesamiento.
- **Procesamiento:** Al presionar el botón "Siguiente", el sistema instancia la lógica de negocio (GeneradorDeEstrategias), activa los sensores simulados y almacena los resultados en la base de datos antes de cambiar de ventana.

8.3. Reporte de Resultados (VentanaResultados.java)

La etapa final presenta la información procesada de manera tabular y ordenada. Esta interfaz recupera los objetos Cultivo ya analizados y expone las decisiones tomadas por el algoritmo inteligente.

- **Tablas de Datos:** Se utilizó un JTable con un DefaultTableModel no editable. Se aplicó un renderizado personalizado (DefaultTableCellRenderer) para estilizar encabezados y celdas, logrando un efecto translúcido sobre el fondo.
- **Información Mostrada:** Temperatura y Humedad detectadas en tiempo real (simuladas según la región), y la **Estrategia de Riego** seleccionada automáticamente por el sistema (Básico, Eficiente o Especializado).



ID Agri.	Cultivo	Región	Temp. (°C)	Humedad (%)	Estrategia Aplica...	Hora Riego
1	arroz	Costa o Chala	24.9	42.1	Riego Eficiente	12:32:58
1	cactus	Costa o Chala	20.9	97.7	Riego Básico	12:32:58
1	lechuga	Costa o Chala	19.0	25.2	Riego Básico	12:32:58
1	platano	Costa o Chala	23.9	13.7	Riego Básico	12:32:58

Ver Informe Detallado Finalizar Riego Cerrar Sesión

9. CONECTIVIDAD A BASE DE DATOS

Dentro del desarrollo del Sistema de Riego Automatizado, se implementó la persistencia de datos mediante una base de datos relacional MySQL. Esta decisión arquitectónica permite garantizar la integridad, seguridad y disponibilidad de la información del agricultor y de los procesos de riego, superando las limitaciones del almacenamiento en archivos de texto plano.

La conectividad entre la aplicación Java y el gestor de base de datos se realizó utilizando la API JDBC (Java Database Connectivity), empleando las siguientes clases e interfaces del paquete java.sql:

- **DriverManager:** Para gestionar los controladores de la base de datos y establecer la conexión.
- **Connection:** Para manejar la sesión activa con la base de datos MySQL.

- **PreparedStatement:** Para ejecutar sentencias SQL precompiladas (INSERT, SELECT), mejorando la seguridad y el rendimiento.
- **ResultSet:** Para almacenar y recorrer los resultados obtenidos de las consultas de lectura.
- **SQLException:** Para el manejo robusto de errores de conexión o consultas fallidas.

El proyecto emplea la base de datos principalmente para dos propósitos fundamentales:

1. Gestión de Identidad y Acceso (Tabla agricultor): Manejada por la clase DatosAgricultor. Se encarga de almacenar la información de registro de los usuarios, asegurando que el DNI sea único en el sistema.

- **Datos almacenados:** ID (Autogenerado), DNI, Nombre, Apellido, Región y Fecha de Registro.

2. Registro Histórico de Operaciones (Tabla historial_riego): Manejada por la clase DatosRiego. Cada vez que el sistema ejecuta un algoritmo de riego en la ventana GestionCultivos, se inserta un registro detallado de la operación, vinculándolo al agricultor mediante una llave foránea.

- **Datos almacenados:** ID Agricultor, Nombre del Cultivo, Temperatura detectada, Humedad detectada, Estrategia aplicada y Hora exacta del evento.

Implementación de la Conexión

La conexión se centralizó en la clase ConexionBD dentro del paquete baseDeDatos, utilizando el patrón *Singleton* para reutilizar el acceso. A continuación, se presenta el fragmento de código utilizado para establecer el enlace con el servidor local:

```

1 package baseDeDatos;
2
3 import java.sql.*;
4
5 public class ConexionBD {
6     private static final String url="jdbc:mysql://localhost/sistema_riego_bd";
7     private static final String user="root";
8     private static final String password="";
9     private static final String driver="com.mysql.cj.jdbc.Driver";
10
11     public static Connection conectar() {
12         Connection conexion=null;
13         try {
14             Class.forName(driver);
15             conexion=DriverManager.getConnection(url,user,password);
16             System.out.println("<< ¡Conexión exitosa a la base de datos! >>");
17         } catch(ClassNotFoundException e) {
18             System.out.println("¡Error! Falta el Driver JDBC de MySQL en el proyecto.");
19         } catch(SQLException e) {
20             System.out.println("¡Error SQL! No se pudo conectar. Verifica usuario/clave.");
21         }
22         return conexion;
23     }
24 }
25

```

phpMyAdmin

Servidor actual: MySQL

Reciente Favoritas

- Nueva
- sistema_riego_bd
 - Nueva
 - agricultor
 - historial_riego

Servidor: MySQL:3306 > Base de datos: sistema_riego_bd > Tabla: agricultor

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Disparadores

Mostrando filas 0 - 4 (total de 5, La consulta tardó 0,0005 segundos.)

SELECT * FROM `agricultor`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

Opciones extra

	id_agricultor	dni	nombre	apellido	region	fecha_registro
☐ Editar ☐ Copiar ☐ Borrar	8	24200181	Alex	SACACO	Yunga	2025-11-27 03:26:06
☐ Editar ☐ Copiar ☐ Borrar	9	1234	a	b	Costa o Chala	2025-11-27 03:38:50
☐ Editar ☐ Copiar ☐ Borrar	10	123456	Angel	Villegas	Puna	2025-11-27 11:21:35
☐ Editar ☐ Copiar ☐ Borrar	11	24200042	Alex	Salazar	Suni	2025-11-27 11:23:34
☐ Editar ☐ Copiar ☐ Borrar	12	24200040	Alex	Salazar	Puna	2025-11-27 11:26:01

Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar

Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista

phpMyAdmin

Servidor actual: MySQL

Reciente Favoritas

- Nueva
- sistema_riego_bd
 - Nueva
 - agricultor
 - historial_riego

Servidor: MySQL:3306 > Base de datos: sistema_riego_bd > Tabla: historial_riego

Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Disparadores

Mostrando filas 0 - 1 (total de 2, La consulta tardó 0,0004 segundos.)

SELECT * FROM `historial\_riego`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

Opciones extra

	id_riego	id_agricultor	nombre_cultivo	temp_detectada	humedad_detectada	estrategia_aplicada	fecha_hora
☐ Editar ☐ Copiar ☐ Borrar	1	10	PAPPA	0.97	6.48	Riego Básico	2025-11-27 11:21:46
☐ Editar ☐ Copiar ☐ Borrar	2	10	Aroz	4.78	64.28	Riego Eficiente	2025-11-27 11:21:46

Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar

Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista

10. REFERENCIAS

- Deitel, P., & Deitel, H. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson.
- Ministerio de Desarrollo Agrario y Riego (MIDAGRI). (2021). *Plan Nacional de Recursos Hídricos del Perú*. Lima, Perú.
- MySQL. (2025). *MySQL 8.0 Reference Manual*. Oracle Corporation.
Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>
- Reese, G. (2000). *Database Programming with JDBC and Java*. O'Reilly Media.